

TP Chapitre 1 : Représentation Numérique

Pistes et Indications

Dr. El Hadji Bassirou TOURÉ
Département de Mathématiques et Informatique
Université Cheikh Anta Diop de Dakar

Ce document fournit des pistes pour résoudre les exercices du TP.

Exercice 1 : Affichage Binaire et Hexadécimal

Piste : print_bits

Pour afficher les bits du MSB au LSB :

- Parcourir de $i = n_bits - 1$ jusqu'à 0
- Pour chaque position, créer un masque : $1ULL \ll i$
- Tester avec AND et afficher '1' ou '0'

```
1 for (int i = n_bits - 1; i >= 0; i--) {  
2     printf("%c", (value & (1ULL << i)) ? '1' : '0');  
3 }
```

Piste : print_hex

Pour afficher les octets en mémoire :

- Caster le pointeur en `unsigned char *`
- Parcourir les octets dans l'ordre des adresses
- Utiliser `%02X` pour 2 caractères hex

```
1 unsigned char *bytes = (unsigned char *)ptr;  
2 for (size_t i = 0; i < size; i++) {  
3     printf("%02X ", bytes[i]);  
4 }
```

Réponse : Pourquoi l'affichage est "inversé" ?

C'est l'effet de l'architecture **little-endian** (x86) :

- L'octet de poids faible est stocké à l'adresse la plus basse
- `0x12345678` est stocké comme `78 56 34 12`
- Ce n'est pas une "inversion", c'est l'ordre de stockage en mémoire

Exercice 2 : Signed vs Unsigned et Extension de Signe

Piste : Différence entre d et e

```
1 uint16_t d = a;           // a = -1 (int8_t)  
2 uint16_t e = (uint8_t)a;
```

Pour d :

1. `a = -1` est d'abord converti en `int` (sign-extension) $\rightarrow 0xFFFFFFFF$
2. Puis converti en `uint16_t` $\rightarrow 0xFFFF = 65535$

Pour `e` :

1. `(uint8_t)a` force d'abord la conversion en 8 bits $\rightarrow 0xFF = 255$
2. Puis zero-extension en 16 bits $\rightarrow 0x00FF = 255$

Piste : Pièges des comparaisons

Règle : quand on compare `signed` et `unsigned`, le `signed` est converti en `unsigned`.

- `-1` converti en `unsigned int` = `0xFFFFFFFF` = valeur maximale
- Donc `-1 < 0U` compare `4294967295 < 0` $\rightarrow$ **faux**

Exercice 3 : Overflow et Détection

Piste : Détection overflow unsigned

Pour `a + b` en `unsigned`, l'overflow se produit si le résultat est plus petit qu'un des opérandes :

```
1 bool will_overflow_add_u32(uint32_t a, uint32_t b) {
2     return (a + b) < a; // ou < b
3 }
```

C'est valide car l'overflow unsigned est défini (wrap-around).

Piste : Détection overflow signed

Pour les `signed`, on ne peut pas utiliser le même test car l'overflow est UB. Il faut tester **avant** l'opération :

```
1 bool will_overflow_add_i32(int32_t a, int32_t b) {
2     if (b > 0 && a > INT32_MAX - b) return true; // overflow positif
3     if (b < 0 && a < INT32_MIN - b) return true; // overflow négatif
4     return false;
5 }
```

Observation : -O0 vs -O2

Avec `-O0` (sans optimisation), le code s'exécute "naïvement" et vous observez le wrap-around.

Avec `-O2`, le compilateur peut supposer que l'overflow signed ne se produit jamais et optimiser en conséquence. Le résultat peut être différent ou inattendu.

Exercice 4 : IEEE 754 et Précision

Piste : Comprendre l'union `float_bits`

L'union permet de "voir" les mêmes 32 bits de trois façons :

- Comme un `float`
- Comme un `uint32_t` (pour afficher en hex)
- Comme une structure avec les champs séparés (signe, exposant, mantisse)

Les bitfields : `23, : 8, : 1` définissent le nombre de bits de chaque champ.

Piste : Valeurs spéciales

- **Zéro** : `exp = 0`, `frac = 0`
- **Infini** : `exp = 255` (tous à 1), `frac = 0`

- **NaN** : $exp = 255, frac \neq 0$

Propriété clé : `NaN != NaN` est toujours **vrai** (c'est la définition de NaN).

Piste : Non-associativité

`(3.14f + 1e10f) - 1e10f` :

1. $3.14 + 10^{10}$: Le float n'a que 7 chiffres de précision. 3.14 est "absorbé" dans l'arrondi.
2. Résultat $\approx 10^{10}$
3. $10^{10} - 10^{10} = 0$ (pas 3.14)

`3.14f + (1e10f - 1e10f)` :

1. $10^{10} - 10^{10} = 0$ (exact)
2. $3.14 + 0 = 3.14$

Exercice 5 : Manipulation de Bits

Piste : Opérations de base

Pour `x = 0xB5 = 10110101` :

- `x & 0x0F` : garde les 4 bits bas $\rightarrow 0x05$
- `x | 0x0F` : met les 4 bits bas à 1 $\rightarrow 0xBF$
- `x ^ 0xFF` : inverse tous les bits $\rightarrow 0x4A$

Piste : Manipulation de bits individuels

- **Tester bit k** : `(x >> k) & 1`
- **Set bit k** : `x | (1 << k)`
- **Clear bit k** : `x & ~(1 << k)`
- **Toggle bit k** : `x ^ (1 << k)`

Piste : Shift arithmétique vs logique

- **Unsigned** : shift logique, des 0 sont insérés à gauche
- **Signed négatif** : shift arithmétique, le bit de signe (1) est répliqué

Pour `0x80000000` :

- `Unsigned >> 4` : `0x08000000` (0 insérés)
- `Signed >> 4` : `0xF8000000` (1 insérés car négatif)

Erreurs courantes

- Oublier ULL pour les shifts de 32+ bits : `1 << 35` est UB, utiliser `1ULL << 35`
- Utiliser `%X` au lieu de `%02X` : affiche "A" au lieu de "0A"
- Comparer des signed et unsigned sans le savoir (warnings ignorés)
- Tester l'overflow signed **après** l'opération (UB)